

# TypeScript Error: Property 'map' does not exist

Cause & Fix — grund-shop project documentation

---

## 1. The Error Message

```
Property 'map' does not exist on type  
'(GetAggregateResult<...>)[ ] | { success: boolean; message: any; }'.
```

```
Property 'map' does not exist on type '{ success: boolean; message: any; }'.
```

TypeScript is reporting that the value being called has a **union type** — it could be one of two different shapes — and `.map()` only exists on one of those shapes (the array). Since TypeScript cannot know at compile time which shape will actually be returned at runtime, it blocks the call until the code proves which branch it is dealing with.

## 2. Root Cause

The server action below has two different return statements with two different shapes:

```
export async function getAllCategories() {  
  try {  
    const data = await prisma.product.groupBy({  
      by: ['category'],  
      _count: true  
    })  
  
    return data;  
    // shape A: { category: string; _count: number }[]  
  
  } catch (error) {  
    return { success: false, message: formatError(error) }  
    // shape B: { success: boolean; message: any }  
  }  
}
```

Because the function can exit through either the `try` block or the `catch` block, TypeScript infers the return type as the **union** of both shapes:

```
{ category: string; _count: number }[] | { success: false; message: string }
```

When this is consumed in the component:

```
const categories = await getAllCategories();  
  
categories.map((x) => ...) // ■ Error
```

`categories` is typed as “array or error object,” and `.map` doesn't exist on the error object branch, so TypeScript refuses to compile until the two cases are distinguished.

**Important:** this is not a false alarm. If `prisma.product.groupBy()` ever throws, the original code would have crashed at runtime trying to call `.map()` on `{ success: false, message: "..."}` . TypeScript caught a real, previously invisible bug risk.

### 3. Fix Option A — Narrow with `Array.isArray()`

Quickest fix. Add a type guard before using the result as an array:

```
const categories = await getAllCategories();

if (!Array.isArray(categories)) {
  // categories is narrowed to { success: false; message: string }
  return null; // or render an error state
}

// categories is now safely typed as the array
categories.map((x) => (
  <Button key={x.category} ...>
    <Link href={`\search?category=${x.category}`}>
      {x.category} ({x._count})
    </Link>
  </Button>
))
```

### 4. Fix Option B — Consistent Wrapper Shape (Recommended)

Better long-term fix. Make both branches return the *same* shape, using a success discriminant on both sides:

```
export async function getAllCategories() {
  try {
    const data = await prisma.product.groupBy({
      by: ['category'],
      _count: true
    })

    return { success: true as const, data };
  } catch (error) {
    return { success: false as const, message: formatError(error) };
  }
}
```

Then in the component:

```
const result = await getAllCategories();

if (!result.success) {
  return null; // or show result.message
}

// result.data is safely typed as the array here
{result.data.map((x) => (
  <Button key={x.category} ...>
```

```
    <Link href={` /search?category=${x.category}`}>
      {x.category} ({x._count})
    </Link>
  </Button>
  )}}
```

This pattern scales better across a codebase like grund-shop: every server action returns `{ success, data }` or `{ success, message }`, so every caller checks the same success field consistently, instead of some call sites using `Array.isArray` and others using a success check.

## 5. Summary

| Approach                                          | Effort            | Best for                                           |
|---------------------------------------------------|-------------------|----------------------------------------------------|
| <code>Array.isArray()</code> guard                | Low               | One-off fix, small components                      |
| Consistent <code>{ success, data }</code> wrapper | Medium (refactor) | Shared action patterns used across many components |

**Key takeaway:** whenever a function's return type can be one of several different shapes (e.g. success vs. error), always check / narrow which shape was returned before calling array methods or accessing shape-specific properties on it.